

DOC_INTEL: API Reference

Stored Procedures

`sp_register_new_files()`

Schema: `DOC_INTEL.DATA`

Purpose: Scans both stages, registers new files, resets retryable failures, promotes abandoned failures, creates new versions for changed files.

Called by: `task_01_register_files` (every 15 minutes)

```
CALL DOC_INTEL.DATA.sp_register_new_files();  
-- Returns: TABLE of pending documents
```

`sp_force_reprocess(p_file_path, p_reason)`

Schema: `DOC_INTEL.DATA`

Purpose: Forces a document to be re-parsed, re-classified, and re-extracted regardless of its current status.

Parameter	Type	Required	Description
<code>p_file_path</code>	VARCHAR	Yes	File path as stored in ingestion_registry
<code>p_reason</code>	VARCHAR	No	Human-readable reason for audit trail

```
CALL DOC_INTEL.DATA.sp_force_reprocess(  
    'documents/my-document.pdf',  
    'Coverage form was misclassified'  
);  
-- Returns: 'OK: File queued for reprocessing as version N' or  
           'ERROR: ...'
```

```
sp_bulk_reprocess(p_folder, p_file_format, p_from_date, p_to_date,  
p_dry_run)
```

Schema: DOC_INTEL.DATA

Purpose: Bulk-queues COMPLETE documents for reprocessing based on filters.

Parameter	Type	Default	Description
p_folder	VARCHAR	NULL	Filter by source_folder (NULL = all folders)
p_file_format	VARCHAR	NULL	Filter by file_format (NULL = all formats)
p_from_date	TIMESTAMP_NTZ	NULL	Filter by first_seen_at >= from_date
p_to_date	TIMESTAMP_NTZ	NULL	Filter by first_seen_at <= to_date
p_dry_run	BOOLEAN	TRUE	If TRUE, shows what would be reprocessed without doing it

```
-- Preview which Claims TIFFs from 2024 would be reprocessed  
CALL DOC_INTEL.DATA.sp_bulk_reprocess(  
  'Claims', 'TIFF',  
  '2024-01-01'::TIMESTAMP_NTZ,  
  '2024-12-31'::TIMESTAMP_NTZ,  
  TRUE  
);  
  
-- Execute  
CALL DOC_INTEL.DATA.sp_bulk_reprocess('Claims', 'TIFF', NULL,  
  NULL, FALSE);
```

```
sp_parse_pending_documents()
```

Schema: DOC_INTEL.DATA

Purpose: Runs AI_PARSE_DOCUMENT on all documents in PROCESSING state. Called by task_02_parse_documents .

```
sp_classify_parsed_documents()
```

Schema: DOC_INTEL.DATA

Purpose: Runs AI_CLASSIFY on parsed documents. Called by task_03_classify_documents .

```
sp_extract_document_attributes()
```

Schema: DOC_INTEL.DATA

Purpose: Runs AI_EXTRACT on classified documents and marks registry as COMPLETE. Called by task_04_extract_attributes .

```
sp_emit_pipeline_metrics()
```

Schema: DOC_INTEL.DATA

Purpose: Emits telemetry summary event and checks alert thresholds. Called by task_05_emit_metrics .

```
sp_approve_override(p_override_id)
```

Schema: DOC_INTEL.ADMIN

Purpose: Approves a pending attribute correction, making it eligible for application in the next Dynamic Table refresh.

```
CALL DOC_INTEL.ADMIN.sp_approve_override('uuid-of-override');  
-- Returns: 'OK: Override approved...' or 'ERROR: ...'
```

```
sp_get_dashboard_stats()
```

Schema: DOC_INTEL.ADMIN

Purpose: Returns key admin dashboard metrics with status indicators (OK/WARNING/CRITICAL).

```
CALL DOC_INTEL.ADMIN.sp_get_dashboard_stats();
-- Returns: TABLE(metric VARCHAR, value VARCHAR, status
                VARCHAR)
```

```
sp_reset_pipeline(p_confirm)
```

Schema: DOC_INTEL.ADMIN

Purpose: Emergency reset — moves all PROCESSING documents back to PENDING.

```
CALL DOC_INTEL.ADMIN.sp_reset_pipeline('CONFIRM_RESET');
-- Must pass exact string 'CONFIRM_RESET' to prevent accidental
   execution
```

Cortex Services

doc_search_svc (Cortex Search)

Document-level semantic search with structured attribute filters.

REST API endpoint:

```
https://<account>.snowflakecomputing.com/api/v2/databases/DOC_INTEL/schemas/SERVICES/c
```

Query parameters:

Parameter	Type	Description
query	string	Natural language search text
columns	array	Columns to return
filter	object	Cortex Search filter object
limit	integer	Max results (default 10, max 1000)

Filterable columns: document_type , doc_status , source_folder ,
line_of_business , coverage_form , ir_policy_number ,
ir_claim_number , file_format , insured_name

SQL example:

```
SELECT PARSE_JSON(SNOWFLAKE.CORTEX.SEARCH_PREVIEW(
  'DOC_INTEL.SERVICES.doc_search_svc',
  '{
    "query": "coverage opinion prior knowledge exclusion",
    "columns":
      ["file_path","document_type","insured_name","doc_summary"],
    "filter": {"@eq": {"source_folder": "Claims"}},
    "limit": 10
  }'
))[ 'results' ] AS results;
```

`doc_page_search_svc` (Cortex Search)

Page-level search for granular content retrieval within multi-page documents.

Filterable: `source_folder` , `ir_policy_number` , `ir_claim_number` , `file_path`

`document_analytics_sv` (Semantic View — Cortex Analyst)

Natural language to SQL for portfolio analytics. See `DATA_DICTIONARY.md` for full column list.

Usage in Next.js app (via `/api/analyst` route):

```
const res = await fetch("/api/analyst", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    question: "How many D&O claims are open by month this year?",
    history: []
  }),
})
const { answer, sql, results, requestId } = await res.json()
// answer:   string   - prose explanation from Cortex Analyst
// sql:      string | null - generated SQL statement
// results:  object[] - executed query results (up to 100 rows)
// requestId: string | null - Cortex Analyst request ID for debugging
```

The `/api/analyst` route calls the Cortex Analyst REST API (`POST /api/v2/cortex/analyst/message`) against `document_analytics_sv` , then executes the generated SQL against Snowflake and returns actual data

rows. The analytics page NL box renders the prose answer, the SQL (expandable), and a results table.

Direct SQL (for Snowflake SQL clients):

```
-- Cortex Analyst is also available via Snowflake Intelligence
      (CoWork)
-- or via direct API call -- no intermediate SQL wrapper needed
SELECT SNOWFLAKE.CORTEX.ANALYST.RUN(
  'DOC_INTEL.SERVICES.document_analytics_sv',
  'How many D&O claims are open by month this year?'
);
```

doc_intelligence_agent (Cortex Agent)

Full routing agent combining document search, page-level search, and Cortex Analyst. Autonomously routes between its three tools based on question intent: - `doc_search` — Cortex Search on full documents (find, retrieve, what does X say) - `page_search` — Cortex Search on individual pages (specific clause language) - `doc_analytics` — Cortex Analyst (how many, count, breakdown, trend)

Usage in Next.js app (via `/api/agent` route):

```
const res = await fetch("/api/agent", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    question: "What exclusions apply to our D&O policies?",
    history: []
  }),
})
// Response is SSE stream (Content-Type: text/event-stream)
const reader = res.body.getReader()
// Each SSE event is JSON with one of these shapes:
// Text chunk:  { delta: string, toolUsed: string, done:
//               boolean }
// Status:      { status: string, toolUsed: string, done:
//               false }
// Chart data:  { resultSet: { columns: string[], rows: any[]
//               [], title?: string }, toolUsed: string, done: false }
// Final event has done: true
```

The `/api/agent` route calls the Cortex Agent REST API (`POST /api/v2/cortex/agent:run`) with SSE streaming. Authentication uses a combined OAuth token (`serviceToken.callerToken`) for SPCS caller identity.

Environment requirements: - `SNOWFLAKE_HOST` env var must be set (available only in SPCS) - Returns HTTP 500 if not running in an SPCS environment

The agent is also available via **Snowflake Intelligence (CoWork)** for direct natural language access without any API calls.

Telemetry Views

`v_pipeline_health`

Current queue state snapshot. One row total.

```
SELECT * FROM DOC_INTEL.TELEMTRY.v_pipeline_health;
-- Columns: pending_count, processing_count, complete_count,
--           failed_count,
--           abandoned_count, skipped_count, total_registered,
--           last_completed_at, oldest_pending_at,
--           max_queue_age_minutes
```

`v_ingestion_metrics`

Daily throughput and failure rates by source folder and format.

`v_cost_by_folder`

Credit consumption by day, source folder, and pipeline stage.

`v_search_analytics`

Query patterns, latency percentiles, and feedback rates.

`v_quality_metrics`

AI classification confidence distribution by document type.

`v_error_summary`

Error patterns grouped by stage and message.

`v_agent_audit_log` **(APP schema)**

Anonymized audit log of all agent/CoWork queries (response text truncated, no PII).

Next.js App API Routes

All routes are deployed as Next.js App Router handlers in `app/app/api/`. Routes marked **requireUser** accept any DOC_INTEL role (user, admin, or pipeline). Routes marked **requireAdmin** require DOC_INTEL_ADMIN specifically.

Route	Method	Auth	Description
<code>/api/health</code>	GET	None	Snowflake connectivity check. Returns <code>{ status, ts }</code> on success, 503 on failure. No database name or error details exposed.
<code>/api/auth</code>	GET	None	Detect current user's DOC_INTEL app role (<code>admin</code> , <code>user</code> , or <code>none</code>). Used by client-side role context.
<code>/api/filters</code>	GET	requireUser	Dynamic lists for filter dropdowns: source folders, LOBs, document types, statuses, file formats, and feature flags.
<code>/api/search</code>	GET	requireUser	Cortex Search hybrid retrieval. Params: <code>q</code> , <code>folder</code> , <code>lob</code> , <code>docType</code> , <code>status</code> , <code>format</code> , <code>limit</code> (max 50). Returns array of results with AI-extracted fields.
<code>/api/agent</code>	POST	requireUser	Cortex Agent (<code>doc_intelligence_agent</code>) SSE streaming endpoint. Requires SPCS environment (<code>SNOWFLAKE_HOST</code>). Body: <code>{ question, history[] }</code> . Returns <code>text/event-stream</code> with events: <code>{ delta, toolUsed, done }</code> , <code>{ status, toolUsed, done }</code> , <code>{ resultSet, toolUsed, done }</code> .
<code>/api/analyst</code>	POST	requireUser	Cortex Analyst REST API against <code>document_analytics_sv</code> . Body: <code>{ question, history[] }</code> . Returns <code>{ answer, sql, results[] , requestId, warnings[] }</code> .
<code>/api/analytics</code>	GET	requireUser	Document distribution chart data (byType, byLob, byDrawer, byStatus).
<code>/api/documents</code>	GET	requireUser	Paginated document catalog browser. Params: <code>folder</code> , <code>docType</code> , <code>lob</code> , <code>status</code> , <code>review</code> , <code>limit</code> (max 200),

Route	Method	Auth	Description
			offset . Returns { rows[], total, limit, offset } .
/api/saved	GET/ POST/ DELETE	requireUser	Saved searches CRUD. POST body: { name, queryText, filtersJson } . DELETE param: ?id= .
/api/bookmarks	POST/ DELETE	requireUser	Document bookmarks. POST body: { filePath, irFileId?, documentType?, insuredName?, note? } . DELETE param: ? file_path= .
/api/feedback	POST	requireUser	Thumbs up/down feedback on search results. Body: { filePath, feedbackType, queryText?, documentType? } . feedbackType must be thumbs_up or thumbs_down .
/api/admin	POST	requireAdmin	Admin actions: force_reprocess , approve_override , reject_override , submit_correction .
/api/config	GET/ PUT	requireAdmin	System config key/value store. PUT body: { key, value } .
/api/quality	GET	requireAdmin	Quality metrics (avgConfidence , lowConfPct , irMatchPct) + pending corrections + feedback log.
/api/registry	GET	requireAdmin	Paginated ingestion registry. Params: status , folder , limit (max 200), offset . QUALIFY deduplication on file_path .
/api/pipeline	GET	requireAdmin	Pipeline queue health snapshot from v_pipeline_health .
/api/tasks	GET	requireAdmin	Snowflake task status via SHOW TASKS + task run history.
/api/users	GET	requireAdmin	User activity from agent_audit_log (last 30 days) + role membership from SHOW GRANTS OF ROLE .
/api/cost	GET	requireAdmin	Credit consumption: by source folder (last 30 days) + by pipeline stage (last 7 days from pipeline_events) . KPIs:

Route	Method	Auth	Description
			<code>total7d</code> , <code>avgDaily</code> (total/30), <code>projectedMonthly</code> .
<code>/api/admin/classification-labels</code>	GET/ POST/ PUT/ DELETE	requireAdmin	Classification labels CRUD: list, create, toggle active, rename/delete categories.
<code>/api/admin/lobs</code>	GET/ POST/ PUT/ DELETE	requireAdmin	Lines of Business CRUD: list, create, toggle active, update details, delete.
<code>/api/models</code>	GET	requireUser	Available Cortex LLM and extraction models. Queries <code>SHOW CORTEX BASE MODELS</code> , filters to GA/PUPR status, returns <code>{ cortex_model: string[], extraction_model: string[] }</code> .
<code>/api/documents/download</code>	GET	requireUser	Generate presigned URL for a document on stage. Params: <code>file_path</code> (required), <code>stage_name</code> (optional, defaults to <code>@DOC_INTEL.DATA.imageright_stage</code>). Enforces <code>canDownload</code> entitlement and LOB access. Logs download event to <code>pipeline_events</code> . Returns <code>{ url, file_path, stage_name }</code> or 404 if file not found.

New Routes Added (Phases 3–6)

Document Detail Routes

Route	Method	Auth	Description
<code>/api/documents/detail</code>	GET	requireUser	Full document detail. Param: <code>id</code> (base64url-encoded <code>file_path</code>). Returns <code>{ document, pendingCorrections[] }</code> . Applies LOB entitlement filter.
<code>/api/documents/history</code>	GET	requireUser	Processing version history. Param: <code>id</code> . Returns <code>{ history[] }</code> with

Route	Method	Auth	Description
			version, status, timestamps, errors.
<code>/api/documents/related</code>	GET	requireUser	Related documents by insured/policy/claim. Param: <code>id</code> . Returns <code>{ related[] }</code> . Applies LOB filter.
<code>/api/documents/export</code>	GET	requireUser	Export filtered document catalog as CSV. Params: <code>format</code> , <code>lob</code> , <code>doc_type</code> , <code>status</code> . Respects <code>canDownload</code> entitlement.

URL scheme: Document `id` params are base64url-encoded file paths (+ → -, / → _, = stripped). Encode: `btoa(filePath).replace(/\+/g, "-").replace(/\/\//g, "_").replace(/=+$/, "")` . Decode: reverse. This is required because Snowflake App Runtime (SAR) cannot deploy Next.js `[param]` bracket directories.

Review Queue

Route	Method	Auth	Description
<code>/api/review</code>	GET	requireUser	Entitlement-scoped review queue. Params: <code>status</code> (pending/approved/skipped/all), <code>limit</code> , <code>offset</code> . Filters by user's <code>lobAccess</code> + <code>docTypeAccess</code> . Returns <code>{ rows[] , total }</code> .
<code>/api/review?action=count</code>	GET	requireUser	Badge count endpoint. Returns <code>{ pending, approved, skipped }</code> scoped to user entitlements.
<code>/api/review</code>	POST	requireUser	Review action. Body: <code>{ filePath, action: "approve" "skip" }</code> . Inserts/updates <code>extraction_overrides</code> with <code>field_name='__review_status__'</code> .

Extraction Test Lab

Route	Method	Auth	Description
<code>/api/admin/extraction-test?action=list_stages</code>	GET	requireUser	Returns configured ingestion stages from <code>ADMIN.ingestion_stages</code> .
<code>/api/admin/extraction-test?action=list_files&stage=<name></code>	GET	requireUser	Lists files on a stage via <code>LIST @stage_name</code> . Filters to supported document extensions.
<code>/api/admin/extraction-test</code>	POST	requireAdmin	On-demand extraction. Body: <code>{ filePath, stageName, mode: "full" "single_attr" }</code> or <code>{ files[], stageName }</code> for batch. Full pipeline: Parse → Classify Document Type → Detect LOB (reads active LOBs from <code>lines_of_business</code> table) → Resolve Schema (COMMON + CATEGORY + LOB_SPECIFIC tiers) → Extract. Returns <code>{ ok, results: { raw_text, classification, lob, extraction, fieldTiers } }</code> .

Classification Labels CRUD

Route	Method	Auth	Description
<code>/api/admin/classification-labels</code>	GET	requireAdmin	List all classification labels ordered by category and sort_order. Returns <code>{ labels[] }</code> .
<code>/api/admin/classification-labels</code>	POST	requireAdmin	Create label. Body: <code>{ label, category }</code> .
	PUT	requireAdmin	Toggle active (<code>{ label, isActive }</code>), rename category (<code>{ action:</code>

Route	Method	Auth	Description
<code>/api/admin/classification-labels</code>			"rename_category", oldName, newName }), or delete category ({ action: "delete_category", category }).
<code>/api/admin/classification-labels? label=<name></code>	DELETE	requireAdmin	Delete a single label by name.

Lines of Business CRUD

Route	Method	Auth	Description
<code>/api/admin/lobs</code>	GET	requireAdmin	List all LOBs ordered by sort_order. Returns { lobs[] } with lob_code , lob_name , description , is_active , sort_order .
<code>/api/admin/lobs</code>	POST	requireAdmin	Create LOB. Body: { lobCode, lobName, description? } . Auto-assigns next sort_order. Returns 409 if code exists.
<code>/api/admin/lobs</code>	PUT	requireAdmin	Toggle active ({ lobCode, isActive }) or update details ({ lobCode, lobName, description }).
<code>/api/admin/lobs? lob_code=<code></code>	DELETE	requireAdmin	Delete a LOB by code.

Routes Updated in Phases 3–6

Route	Change
<code>/api/health</code>	Now requires <code>DOC_INTEL</code> role (was unauthenticated — security fix)
<code>/api/documents/download</code>	Enforces <code>canDownload</code> + LOB access. Logs download event to <code>TELEMETRY.pipeline_events</code> .
<code>/api/documents/proxy</code>	Enforces <code>canDownload</code> + LOB access. Logs view event to <code>TELEMETRY.pipeline_events</code> .
<code>/api/search</code>	New <code>scope</code> param (base64url <code>file_path</code>) switches to <code>doc_page_search_svc</code> for intra-document search.
<code>/api/agent</code>	Now uses Cortex Agent REST API with SSE streaming. Requires <code>SPCS</code> (<code>SNOWFLAKE_HOST</code>). Citations formatted as <code>[filename, p.N]</code> .
<code>/api/admin/pipeline</code>	New <code>action=preview_reprocess</code> returns match count without triggering reprocess.
<code>/api/bookmarks</code>	Added GET handler to list current user's bookmarks from <code>ADMIN.document_bookmarks</code> .

Entitlements Architecture (Multi-Role)

Overview

Access control is role-based with multi-role support. Users get access exclusively through role membership — no custom/direct entitlements. A user can be assigned multiple roles; effective access = union of all assigned roles' arrays.

Tables

Table	Purpose
<code>ADMIN.entitlement_roles</code>	Named role templates with <code>menu_access</code> , <code>lob_access</code> ,

Table	Purpose
	<code>doc_type_access</code> , <code>can_download</code> arrays
<code>ADMIN.user_entitlements</code>	User rows with <code>is_active</code> flag (app role is derived from entitlement roles' <code>menu_access</code>)
<code>ADMIN.user_role_assignments</code>	Junction table: (<code>user_name</code> , <code>role_id</code>) — multiple roles per user
<code>ADMIN.lines_of_business</code>	Master LOB configuration table — drives AI_CLASSIFY LOB detection dynamically. Managed via Admin → Lines of Business

Resolution Logic (`app/lib/entitlements.ts`)

1. Load all active role assignments for the user from `user_role_assignments`
2. For each role, load `menu_access` , `lob_access` , `doc_type_access` , `can_download`
3. UNION all arrays — effective access = union of all assigned roles
4. If ANY array contains `"*"` (wildcard), that dimension grants full access
5. `canDownload = true` if ANY assigned role allows downloads
6. `appRole` is derived: if any `menuAccess` entry starts with `admin:` , the user is `admin` ; otherwise `user`

SQL Helper Functions

```
// Apply to any SQL query to enforce LOB entitlements
lobFilter(entitlements, "line_of_business")
// → AND line_of_business IN ('DO','EO','CY') | (empty string if wildcard)

docTypeFilter(entitlements, "doc_type_category")
// → AND doc_type_category IN ('FNOL','Examiner Report') | (empty string if wildcard)
```

Both helpers return an empty string when the user has wildcard access, so they can be unconditionally appended to WHERE clauses.

Sidebar Reactivity

After an admin modifies role assignments, the sidebar menu updates without a page reload: - `RoleProvider` exposes `refreshAuth()` via

`useRole()` hook - The user management page calls `refreshAuth()` after saving - Re-fetches `/api/auth`, which re-evaluates entitlements and updates the sidebar

Last updated: August 2026 — Phases 3–9 complete, plus CRUD labels/LOBs, Test Lab pins, dynamic LOB detection